

[复习主页](#) · [真题与考点](#) · [课程计划](#)

JPEG 完整流水线

这是那道 15 分的大计算题。前几节我们已把 $RGB \rightarrow YCbCr$ 、DCT、量化拆开练过;本节把最后三步——之字扫描、DC 差分、AC 游程——一次串成完整答卷,给你一套照抄就能得分的解题模板。

直觉类比

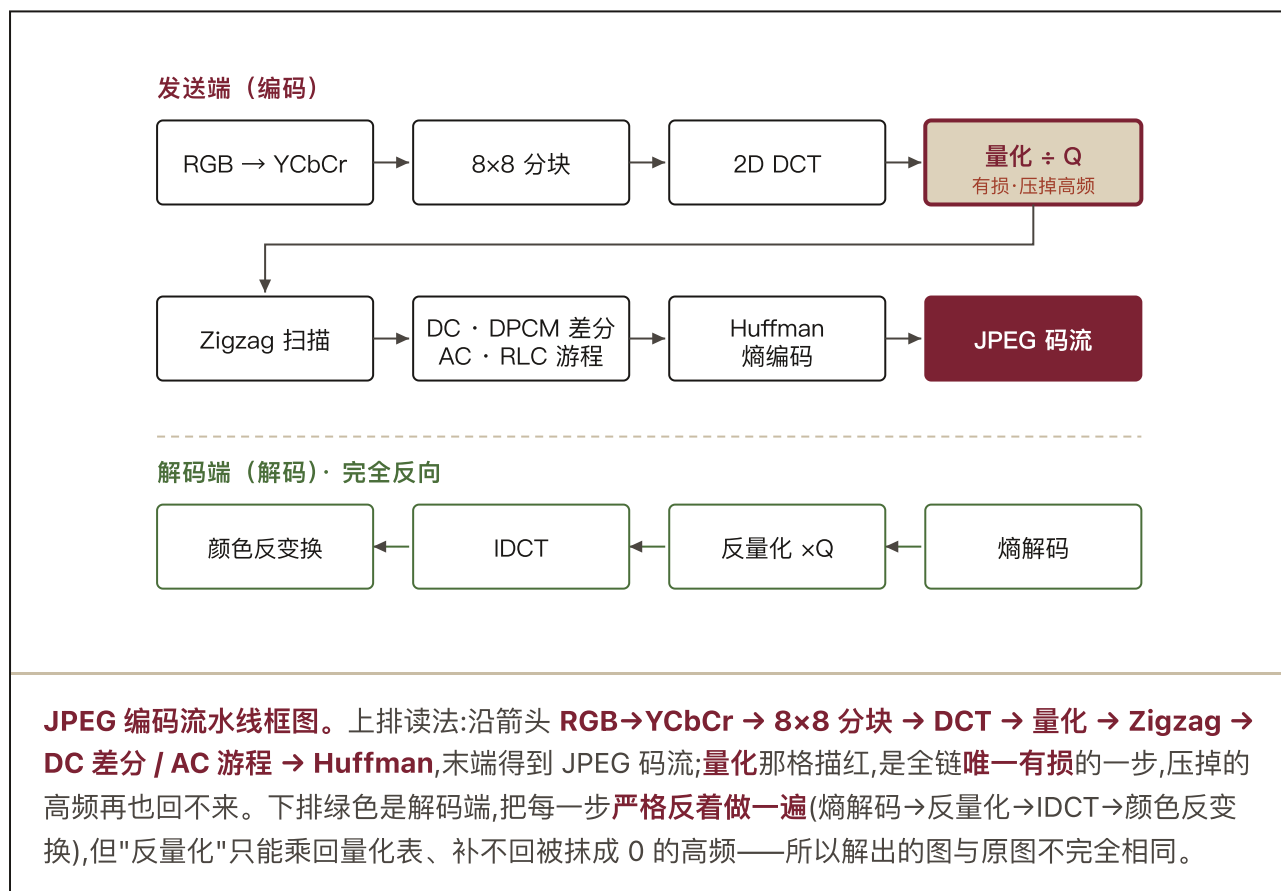
量化后的 8×8 系数块像一张左上角写满字、右下角几乎空白的表格:能量都堆在左上(低频),右下角全是 0。若逐行读,0 会被一堆非零数打散;而之字(Zig-zag)扫描是沿对角线来回走,把"从最低频到最高频"顺序拉直——于是所有 0 自然滑到队尾,连成一条长长的零尾巴。后面的游程编码就靠这条尾巴"一句话交代完",这正是 JPEG 省空间的关键一招。

1. JPEG 五步流水线

把整条压缩链背下来,大题第一问常直接考"写出 JPEG 编码步骤"。真题的计算重心在第 4、5 步,但前三步的名词不能漏。

步	操作	做了什么 / 为什么
1	RGB → YCbCr + 色度二次采样	分离亮度 Y 与色度 Cb/Cr;人眼对色度不敏感,4:2:0 先扔掉 3/4 色度信息。
2	分 8×8 块做 2D DCT	把空间像素变到频域,能量集中到少数低频系数(左上角)。
3	量化 $\hat{F}(u,v)=\text{round}(F(u,v)/Q(u,v))$	除以量化表并取整,高频被压成 0——JPEG 有损、压缩的主来源。
4	之字扫描 + 游程编码	Zig-zag 拉直系数;DC 用 DPCM 差分,AC 用 (run,level) 游程。本节重点。
5	熵编码(Huffman)	对 DC 差分和 AC 游程符号做变长编码,进一步无损压缩。

记忆口诀:换色 → 变频 → 量化 → 扫描游程 → 熵编码。前三步"有损",第 4、5 步"无损"。



2. 四个关键定义

Zig-zag 之字扫描

沿对角线来回,把 8×8 系数块按频率由低到高展开成 64 个数的一维序列。*目的:* 让左上低频集中在前、右下零系数集中在后,拉长零游程,喂给游程编码。

DC 系数与 AC 系数

DC = $F(0,0)$,块的平均亮度(直流分量);**AC** = 其余 63 个系数(交流分量)。二者分开编码。

DC 的 DPCM 差分

相邻块 DC 值接近,故只存差值 $d = DC_{\text{本块}} - DC_{\text{前块}}$ (首块 $d = DC$ 本身)。

Differential PCM,差分脉冲编码,用小数代替大数。

AC 的游程编码 RLC

把 AC 序列写成一串 **(RUNLENGTH, VALUE)**:VALUE 前面有几个连续 0(run),VALUE 是该非零值。末尾若全为 0,用 **(0,0)=EOB**(块结束)一次收尾。

3. 为什么先做之字扫描

量化后的块长这样(下面例题会用到它):能量堆在左上,右下大片 0。假如逐行读,第一行读完非零值后马上撞上 0,下一行又冒出非零值,0 被切成好几段短游程——游程编码几乎占不到便宜。

DCT 系数 F				量化表 Q				量化结果 $\hat{F}$			
240	-18	5	1	16	11	16	24	15	-2	0	0
-12	9	-2	0	12	14	22	40	-1	1	0	0
4	-3	1	0	14	22	40	57	0	0	0	0
1	0	0	0	24	40	57	80	0	0	0	0

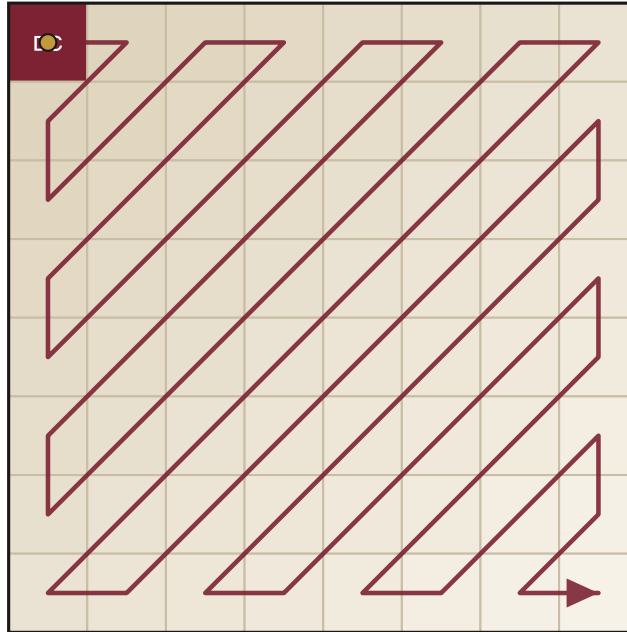
高频区大量变 0

量化:高频被抹平成 0(有损的根源)。逐格计算 $F(u,v)=\text{round}(F(u,v)/Q(u,v))$:量化表 Q 的值**越往右下越大**(高频),而那里的 DCT 系数本就小,一除就四舍五入成 0。于是左块"满是数字"被压成右块"只剩左上角几个非零值 + 一大片 0"。这些 0 无法在解码时恢复——这是 JPEG 唯一不可逆的一步,量化表调得越狠,零越多、压缩率越高、画质越糊。(图示用 4×4 示意,考试实际是 8×8。)

之字扫描换了个走法:它按"离左上角的对角线距离"由近及远地遍历,而对角线距离恰好对应 DCT 的频率高低。于是低频非零值全排在前面,一旦进入高频区就是连续的 0,直到块尾。一条长零尾巴用一个 EOB 就能表达完毕——这就是 Zig-zag 存在的唯一目的:把二维的"零分布"重排成一维的"零聚集",最大化零游程长度。

记住扫描的头几步走向: $(0,0) \rightarrow (0,1) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (1,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow \dots$,像沿反斜线来回折返。这个顺序背下来,考场上照着块填一维序列即可。

低频起点 (0,0)



高频终点 (7,7)

Zig-zag 之字扫描路径。从左上金点 **DC(0,0)** 出发,沿对角线来回折返,一路走到右下角 (7,7),把 8×8 的二维系数拉成 **64 个数的一维序列**。格子底色由深到浅代表频率由低到高——折线恰好**按频率从低到高**穿过所有格子。怎么读:线经过的先后顺序就是写进一维序列的顺序。为什么这样走:低频非零值集中在左上、被先取走,高频区的 0 自然排到队尾连成一长串,给游程编码留出一个 EOB 就能收尾的"零尾巴"。

某图像块经 DCT 与量化后得到下面的 8×8 量化系数块。已知前一个块的 DC 系数为 **38**。请:①做之字扫描得一维序列;②对 DC 做 DPCM 差分;③对 AC 做游程编码 (run, level);④说明如何对 AC 结果做 Huffman 熵编码。

40	5	-1	0	0	0	0	0
-3	2	0	0	0	0	0	0
0	0	0	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

红 = DC(0,0);深色格 = 非零 AC;浅色 0 是被量化压掉的高频。

- ① 之字扫描,展开成一维。沿 Zig-zag 顺序取值: (0,0)=40 → (0,1)=5 → (1,0)=-3 → (2,0)=0 → (1,1)=2 → (0,2)=-1 → (0,3)=0 → (1,2)=0 → (2,1)=0 → (3,0)=-1,其后位置全是 0。

40, 5, -3, 0, 2, -1, 0, 0, 0, -1, 0, 0, ..., 0 (共 64 个,后 54 个全 0)

第 1 个是 DC=40,后面 63 个是 AC。注意零已被扫描聚拢到尾部,这正是之字扫描的功劳。

序号	0(DC)	1	2	3	4	5	6	7	8	9	10~63
值	40	5	-3	0	2	-1	0	0	0	-1	全 0

- ② DC 用 DPCM 差分。本块 DC = 40,前块 DC = 38,故差值 $d = 40 - 38 = 2$ 。JPEG 只把这个 **2** 送去编码,而不是 40——相邻块亮度接近,差值远小于原值,更省位数。

DC 编码值 = 2(若为首块则直接编码 40)。

3 AC 用游程编码 (run, level)。对 AC 序列 5, -3, 0, 2, -1, 0, 0, 0, -1, 0...0 从左到右扫,记录每个非零值前有几个连续 0:

非零值	前导 0 个数 (RUN)	(RUN, LEVEL)
5	0	(0, 5)
-3	0	(0, -3)
2	1	(1, 2)
-1	0	(0, -1)
-1	3	(3, -1)
—(其后全 0)	—	(0, 0) = EOB

AC 游程结果:(0,5) (0,-3) (1,2) (0,-1) (3,-1) (0,0)

尾部 54 个 0 一个 EOB (0,0) 全部交代完——这就是零尾巴带来的压缩红利。

4 对 AC 结果做 Huffman 熵编码。每个 AC 符号进一步写成 (RUNLENGTH, SIZE, AMPLITUDE):SIZE = 表示 |level| 所需的位数,AMPLITUDE 是该值的二进制码(负数取正数的反码,以 0 开头即为负)。

(RUN, LEVEL)	SIZE	AMPLITUDE	(RUN, SIZE, AMP)
(0, 5)	3	101	(0, 3, 101)
(0, -3)	2	00	(0, 2, 00)
(1, 2)	2	10	(1, 2, 10)
(0, -1)	1	0	(0, 1, 0)
(3, -1)	1	0	(3, 1, 0)
(0, 0) EOB	—	—	(0, 0)

规则:把 **(RUNLENGTH, SIZE)** 拼成一个字节(各 4 位)去查 Huffman 表得变长码,AMPLITUDE 直接附在后面原样发送(不再 Huffman)。DC 那边则用 (SIZE, AMPLITUDE): $d=2 \rightarrow \text{SIZE}=2$ 、 $\text{AMP}=10 \rightarrow (2, 10)$,SIZE 查 DC 专用 Huffman 表。

最终码流 = DC 的 Huffman 码 + 各 AC 的 Huffman 码,即完成 JPEG 一个块的全部编码。

自测题

某 8×8 块之字扫描后的一维序列为(第一个数为 DC):

28, 3, 0, 0, -2, 0, 0, 0, 0, 4, 0, 0, ..., 0 (其后全为 0)。

请写出对 AC 部分的游程编码 (run, level),并以 EOB 收尾。下列哪一项正确?

- A. (0,3) (2,-2) (4,4) (0,0)
- B. (1,3) (2,-2) (4,4) EOB
- C. (0,3) (3,-2) (5,4) (0,0)
- D. (0,3) (2,-2) (4,4) (1,0)

▼ 答案与解释

答案:A (0,3) (2,-2) (4,4) (0,0)

去掉首位 $\text{DC}=28$,AC 序列为 3, 0, 0, -2, 0, 0, 0, 0, 4, 0...0。逐个非零值数它前面的 0:

- **3**:前面 0 个 0 $\rightarrow (0, 3)$
- **-2**:前面 2 个 0 $\rightarrow (2, -2)$
- **4**:前面 4 个 0 $\rightarrow (4, 4)$
- 其后全 0 $\rightarrow (\mathbf{0}, \mathbf{0}) = \mathbf{EOB}$

B 把第一个 run 数错(3 前面没有 0,不是 1);C 把中间零个数多算了;D 结尾必须是 (0,0),不是 (1,0)。

易错提醒:run 计的是该非零值之前紧邻的连续 0 个数,不是从块头累计;EOB 恒为 (0,0)。

下一步 → JPEG 处理的是单张图。视频还要吃掉帧与帧之间的时间冗余——进入 0017 · 运动补偿 · 运动矢量 MV 与 MAD, 拆解第二类 15 分计算题: 块匹配求运动矢量。